Disease Outbreak Risk Classification using KNN

AI & ML Lab (CCE3131) Mini-Project #6 — Manipal University Jaipur

1. Aim and problem statement

Classify a patient's symptom vector into **Low / Medium / High** outbreak-risk so that high-risk patients can be flagged early for screening. The model is K-Nearest Neighbours (KNN): a new patient is given the risk class that most of the most-similar past patients had.

2. Plain-English idea of KNN

KNN has no training in the usual sense; it just stores the data. To predict for a new patient it (1) measures how similar that patient is to every stored patient, (2) picks the **k** closest, and (3) lets them vote. Small k = follows noise (over-fit); large k = too blurry (under-fit). Because 'closest' depends on distance, **features must be on the same scale**.

3. Dataset

Used here: a reproducible synthetic dataset (3000 patients, 18 features): 14 symptom flags, age, temperature, days since onset, contact with a known case. Each risk class has its own symptom profile, deliberately overlapping so the task is not trivial. Class split is about 50% Low, 30% Medium, 20% High.

Important honesty note: because the data is synthetic, the scores below show that the pipeline works, not how accurate KNN would be on real patients. The suggested dataset in the lab list is 'Synthetic/UCI symptom dataset', so synthetic is allowed, but state this clearly in your report/viva. To use a real CSV: `python outbreak_knn.py --csv yourfile.csv --target risk`.

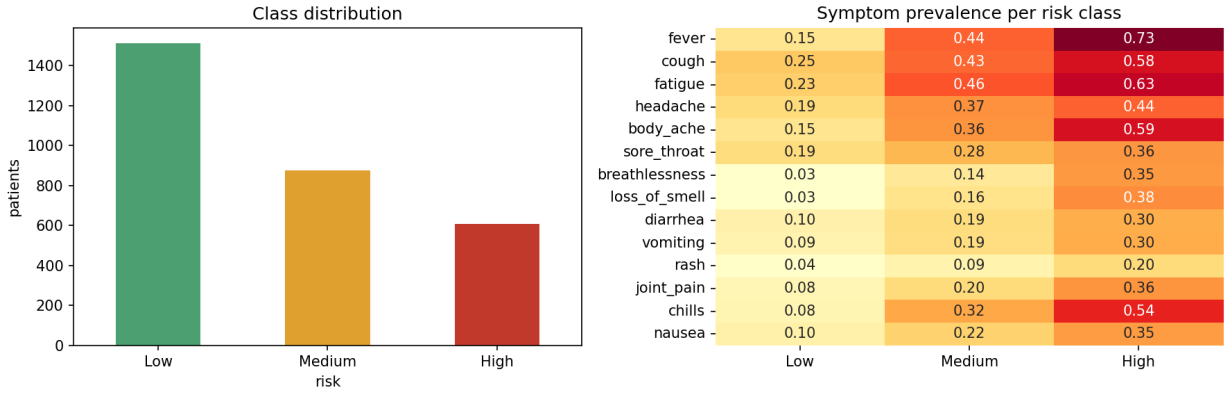


Figure 1: class distribution and symptom prevalence per class

4. Method (what the code does)

- **Split:** 80/20 stratified train/test (2400 / 600), fixed random seed.
- **Scaling:** StandardScaler fitted on training data only (inside a Pipeline, so no leakage into cross-validation).
- **Choose k:** 5-fold cross-validated macro-F1 for k = 1..40 (Figure 2).
- **Grid search:** k (odd values 3-31), weights (uniform / distance), metric (euclidean / manhattan / cosine).
- **Evaluate once** on the untouched test set: accuracy, macro-F1, per-class recall, confusion matrix.

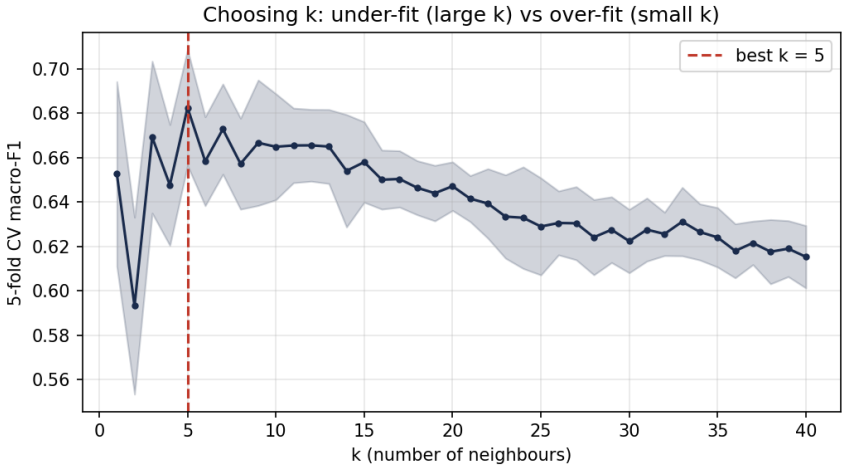


Figure 2: CV macro-F1 versus k (euclidean, uniform weights). Very small k is noisy; large k slowly degrades.

5. Results

Item	Value
Baseline k=5, NO scaling (test accuracy)	0.665
Baseline k=5, WITH scaling (test accuracy)	0.728
Best grid-search params	k=25, weights=distance, metric=cosine
Best CV macro-F1 (train folds)	0.719
Test accuracy (tuned model)	0.788
Test macro-F1 (tuned model)	0.745

Class	Precision	Recall	F1	Support
Low	0.81	0.97	0.89	303
Medium	0.73	0.47	0.57	175
High	0.77	0.79	0.78	122

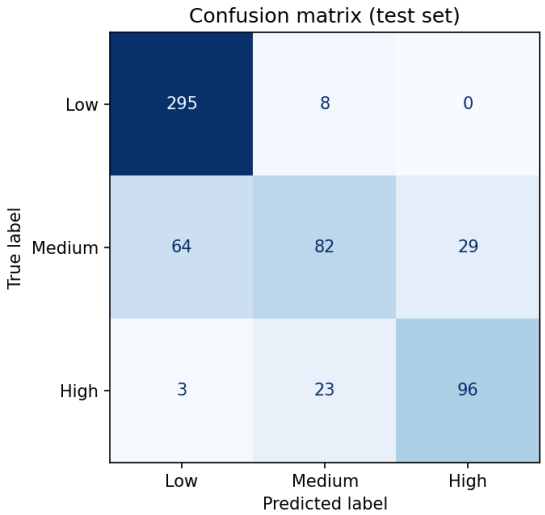


Figure 3: confusion matrix on the 600 test patients

6. Interpretation (what to say in the viva)

- **Scaling matters:** accuracy rose from 66% to 73% just by standardising, because age and temperature otherwise dominate the distance over 0/1 symptom flags.
- **Tuning helped further:** the tuned model reaches 79% test accuracy.
- **Weak spot is the Medium class** (recall 47%). Medium patients overlap with both Low and High, so neighbours disagree. High-risk recall is 79%; for a screening tool, missed High cases are the costliest error, so this is the number to improve first (e.g. class weights, a lower decision threshold for High, or more features).
- **Why accuracy alone misleads:** Low is 50% of the data, so a model that over-predicts Low looks fine on accuracy. That is why macro-F1 and per-class recall are reported.

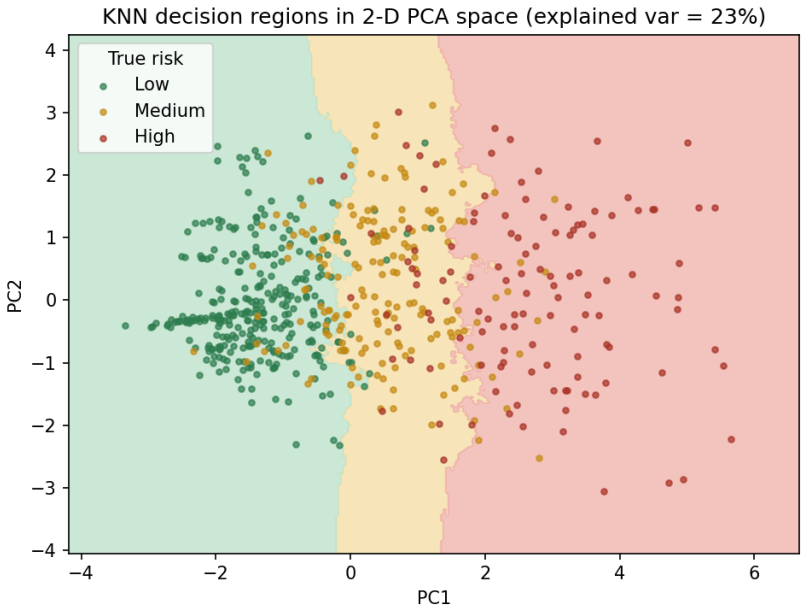


Figure 4: KNN regions in a 2-D PCA projection (only 23% of variance, so classes look more mixed than they are in full feature space; for visualisation only).

Demo prediction: a 58-year-old with fever, cough, breathlessness, loss of smell, fatigue, 38.9 °C, 4 days since onset, contact with a case → predicted **High** (probabilities: {'Low': 0.0, 'Medium': 0.218, 'High': 0.782}).

7. Limitations

- Synthetic data: real clinical data has missing values, noisy reporting and different class balance.
- KNN is slow at prediction time on large data (it compares against every stored patient) and suffers in high dimensions.
- Not a medical device; outputs are for coursework and early-screening demonstration only.

8. Possible extensions (pick one if you want extra marks)

- Compare with Logistic Regression / Decision Tree on the same split (your list has #92, #60-style comparisons).
- Add `class_weight`-style handling: oversample High and Medium (SMOTE) and re-measure recall.
- Replace synthetic data with a real symptom dataset and re-run the same script unchanged.

9. Likely viva questions

Q. Why KNN is called a 'lazy learner'?

It does no model fitting; it stores the data and does all the work at prediction time.

Q. Why scale features?

KNN uses distance; unscaled large-range features (age, temperature) would overpower 0/1 symptom flags.

Q. How did you choose k?

5-fold cross-validated macro-F1 over k=1..40, then grid search with metric and weighting.

Q. What happens for k=1 vs very large k?

k=1 over-fits (noisy, high variance); very large k under-fits (blurs classes, high bias).

Q. Why macro-F1 instead of accuracy?

Classes are imbalanced; macro-F1 averages per-class F1 so small classes count equally.

Q. Why is Medium worst?

Its symptom profile overlaps both Low and High, so nearest neighbours are mixed.

Q. Which error is worst here?

Predicting Low for a truly High patient (missed outbreak case), so High recall matters most.

Q. Why cosine / distance weighting?

Cosine compares symptom patterns rather than magnitudes; distance weighting lets closer neighbours count more.

Q. Is your data real?

No, synthetic and reproducible (seed 42), designed with overlapping class profiles; the pipeline also runs on a real CSV.

Q. How would you improve it?

Real data, class balancing, feature selection, and comparing against other classifiers.

10. How to run

```
pip install scikit-learn pandas numpy matplotlib seaborn
python outbreak_knn.py
(figures and results.json are written to ./outputs/)
```